

Autonomous Machine Learning Pipeline: An End-to-End Holistic Framework for Automated Problem Discovery and Model Deployment

Punya Mittal

School of Computer Science and Engineering
Vellore Institute of Technology, Chennai
Campus
Chennai, India
punya.m215@gmail.com

Garv Bansal

School of Computer Science and Engineering
Vellore Institute of Technology, Chennai
Campus
Chennai, India
garvbansal20066@gmail.com

Vaibhav Raj

School of Computer Science and Engineering
Vellore Institute of Technology, Chennai
Campus
Chennai, India
vaibhav28nov@gmail.com

Trilok Nath Pandey

School of Computer Science and Engineering
Vellore Institute of Technology, Chennai
Campus
Chennai, India
triloknath.pandey@vit.ac.in

Abstract—The definition of actionable machine learning problem statements and the procurement of quality and relevant datasets remain the most time consuming, human-labor intensive bottlenecks of the present-day artificial intelligence (AI) workflow. Conventional methodologies get bogged down by manual requirement specification elicitation, technical audits, data procurement etc, which typically took days or weeks from the business level goal down to a ready-to-deploy baseline model. We propose here the "Autonomous ML Automation Pipeline," a multi-agentic framework to resolve these bottlenecks by automatically bridging the gap between direct natural language prompt and production ready software repository. This work is the first to move away from large socio-technical scrapes of previous systems and is driven by a User Directed Problem Definition orchestrator that acts as a cognitive bridging mechanism that translates vague human intention into clear technical specifications. In its architecture it involves a multi-gate high fidelity decision agent to jointly perform ethical, causal, and technical verification and uses a Semantic Dataset Discovery system powered by deep-learning based vector embeddings to find applicable data sources to the problem statement. The accuracy of the system was validated experimentally with an existing set of 33 unique problem statement tests, covering churn prediction, medical diagnosis assistance and financial fraud prediction. It achieved an average model accuracy and R-squared of 0.690, precision of 0.782, and F1-score of 0.746 while performing the task with drastically reduced end to end latency when contrasted with human workflows.

Keywords —AutoML, Prompt-Based ML, Decision Agents, Causal Inference, Semantic Search, Autonomous Systems, GitHub Automation, LLM Orchestration, Responsible AI.

I. INTRODUCTION

AI has experienced wide-scale democratization, leading to the paradigm shift of modern software, as data-driven predictive models become more deeply embedded in business logic rather than acting as an afterthought. Yet even with powerful AutoML trainers, a massive 'Cold Start' bottleneck persists. A data science project involves identifying a problem, defining target variables, and obtaining necessary data—a process which is still extremely

manual, laborious, and prone to errors, and necessitates human expertise. Translating a fuzzy business problem like 'we need to reduce churn' to a solid ML specification requires domain knowledge and weeks of iterative effort.

According to recent industry survey statistics, a shockingly high 80% of ML projects do not get into production [4]. This failure is primarily attributed to the disconnect between the human desire for a business solution and the available underlying data technical constraints. Beyond this, many systems erroneously attempt to apply predictive black-box heuristics to problems which are fundamentally causal in nature. When dealing with sensitive areas such as medicine, finance, or public policy, an inability to distinguish correlation from causation can result in biased or outright wrong decision-making systems.

To mitigate this gap, we propose an Autonomous ML Automation Pipeline. Our solution incorporates a state-of-the-art User-Driven Prompting Strategy, allowing a non-expert user to express their high-level objective using natural language. An important component of our framework is the Multi-Gate Decision Agent, which provides a structured methodology for validating each proposed issue through four distinct phases: Intent Classification, Feasibility Auditing, Causal Validity, and Heuristic Justification.

The core contributions of this study are as follows:

1) Context-Aware Problem Canonicalization: A mechanism for translating vague user prompts into structured ML specifications.

2) Multi-Gate Validation Layer: A filtering system that prevents unethical or unfeasible model deployments by distinguishing between predictive and causal tasks.

3) End-to-End Orchestration: A fully automated pipeline that generates production-ready GitHub repositories with significantly reduced end-to-end latency compared to manual workflows.

II. LITERATURE SURVEY

AutoML has progressed significantly over the past decade; evolving from a mere hyperparameter optimization problem to the full pipeline automation. Some of the groundbreaking work like Auto-Sklearn [1] and Optuna [2] have effectively automated the model selection and ensembling steps. However, a crucial 'Pipeline Myopia' persists-these systems are able to work only if they are provided a curated dataset and a clearly defined target variable by the human user, leaving the pre-training phase entirely untouched.

With the explosion of LLM's, the automation of software has entered a new paradigm. Llama 3 and GPT-4 [3] have exhibited groundbreaking capabilities in translating an unstructured natural language command to executable code, building upon transformer architectures [20]. Nevertheless, their use in the mathematically sensitive field of ML have proven to lead to 'syntactically correct, statistically incorrect code. The training scripts generated do not properly account for severe class imbalance or inadvertently include temporal data leak, or failure to account for feature scaling between training and inferencing time.

To combat this problem we introduce the 'Agent-in-the-Loop' framework. The LLM in the Agent-in-the-Loop framework does not simply produce a static script. Instead, the LLM acts as a Cognitive Orchestrator that controls a set of specific technical sub-agents-FeasibilityClassifier, DatasetMatcher, and AutoMLTrainer, all built with hard-coded statistical safeguards [3]. Beyond hyperparameter optimization there has been a lot of work on automating the end-to-end ML workflow; TPOT [13] and Auto-WEKA [19] uses genetic programming to optimize an ML pipeline, whereas H2O AutoML [14] provides a stacking mechanism over hundreds of diverse models. These methods are still lacking since they still expect a human curated dataset, which doesn't address the data discovery problem.

Our system automates the pre-training phase via semantic search. Various works on data lake search [15] and metadata driven retrieval have partially addressed the issue of dataset discovery. However these methods are applicable to enterprise data warehouse scenarios, not open domain natural language problem statements. Our Semantic Dataset Discovery Engine bridges this gap by using sentence-transformer embeddings [16] to match natural language problem statements to Kaggle and Hugging Face datasets.

Domain specific AutoML's for healthcare [17] and finance [18] have yielded state-of-the-art results, but are not generalizable to problems outside their respective domains. Our system is domain-agnostic and handles classification, regression, and text problems without manual adaptation; which was validated using our benchmark suite consisting of wine quality prediction, fraud detection and spam classification.

Unlike general purpose LLM's, like GPT-4 [18] and Llama 3, that generate the code, our framework treats the LLM as a reasoning layer, not a code generator. This difference is vital as LLM's are prone to ignoring class imbalance, data leakage and feature alignment problems. Our Multi-Gate Decision Agent addresses this issue by imposing constraints programmatically ensuring the

produced solution meets at least the bare minimum statistical requirements.

III. METHODOLOGY

A. System Architecture and Orchestration Logic

The Autonomous ML Automation Pipeline is structured as an asynchronous agent-driven workflow consisting of five distinct operational phases:

- 1) *Problem Interface & Canonicalization: Accepts raw, high-level user prompts. An LLM-based agent parses the intent to generate a structured JSON specification by identifying the probable target variables, suggested features, and task types.*
- 2) *Multi-Gate Decision Agent: The central brain of the pipeline where canonicalized tasks undergo a strict four-stage verification procedure (Intent, Feasibility, Causal, Justification).*
- 3) *Semantic Dataset Discovery Engine: Utilizes sentence-transformers/all-MiniLM-L6-v2 to represent the query as high dimensional vector and compares to pre-indexed Kaggle and HuggingFace dataset metadata by calculating Cosine Similarity using traditional NLP techniques [22].*
- 4) *AutoML Ensemble Engine: Fully automated data clean-up, detects, and impute missing data intelligently, running an exhaustive search over an ensemble of gradient-boosted trees XG Boost [25] and linear models.*
- 5) *Code Synthesis & Publishing Engine: Automatically generates and pushes a complete production-ready repository on GitHub with train.py, predict.py, requirements.txt and README.md.*

B. The Multi-Gate Decision Agent

The Decision Agent applies a stringent four phase filtering logic to avoid "Garbage-In, Garbage-Out" (GIGO) failure modes of the pipeline:

Gate 1 — Intent Classification: Utilizes a fine-tuned classifier to differentiate between predictive queries ("predict house prices") and open-ended explorations.

Gate 2 — Technical Feasibility Audit: The agent is able to figure out whether a concrete target variable and necessary inputs are derivable from the task.

Gate 3 — Causal Validity Gate: This filter is for policy manipulation, or high stake decision where correlation is risky (e.g. Medical Triage). This will guide user to causal discovery systems built on Pearl's do-calculus [3].

Gate 4 — Heuristic Justification: Verifies that a machine learning solution provides a tangible advantage over simple threshold-based business rules.

C. Semantic Matching Equation

The core matching equation uses the Cosine Similarity between problem embeddings (E_p) and dataset metadata (E_d):

$$S = (E_p \cdot E_d) / (\|E_p\| \|E_d\|)$$

(1)

where S is the similarity score used to rank the datasets from Hugging Face and

Kaggle. A threshold of $S \geq 0.60$ was required for the datasets to be accepted for analysis.

D. Autonomous Pipeline Orchestration Algorithm

The pipeline executes the following steps:

- 1) *The configuration and API keys (Kaggle and GitHub) were initialized.*
- 2) *Accepts user prompts describing high-level ML goals.*
- 3) *Apply Intent Classification — identify whether predictive.*
- 4) *Apply Feasibility Check — identify target/features.*
- 5) *Apply Causal Validity Gate — block high-stakes/intervention tasks.*
- 6) *Apply Justification Filter — verify $ML > Rules$.*
- 7) *If approved, canonicalize the task into the JSON specification.*
- 8) *Search Hugging Face/Kaggle for datasets using Cosine Similarity.*
- 9) *The top-ranked dataset was downloaded and auto-cleaning was performed.*
- 10) *Train via AutoML (Scikit-learn [23]/PyCaret) and select the best models.*
- 11) *The repository files were generated and published on GitHub.*

Figure 1. illustrates the complete end-to-end workflow of our proposed pipeline. The orchestrator manages the state transitions between each phase, with explicit success and failure boundaries enforced after each stage. If there is a failure at any point (a dataset discovery miss, a rejection based on feasibility, etc.), the pipeline terminates gracefully and logs the reason for this failure so that a diagnosis and solution can be identified systematically. As such, only validated ML solutions that have a high degree of confidence will be carried on to the code generation and deployment stage.

IV. DATASET AND TRAINING SETUP

A. Dataset description and validation strategy

The benchmark we used to test the system performance is composed of 33 distinct and varied ML problems which were specified through natural language and processed automatically by the pipeline

- Total Runs: 33 end-to-end execution runs.
- Unique Trained models: 19 models deployed successfully (post multi-gate filter).
- Validation depth: Models were validated with an 80/20 split train-test dataset retrieved by the system via semantic search with rigid feature alignment logic.
- Model diversity: Tasks include classification (sentiment analysis, churn), regression (house prices) and text classification.

To further assess generality, 5 case studies on 5 standard research benchmark datasets, namely UCI SMS Spam Collection [7], ULB Credit Card Fraud dataset [8], UCI Wine Quality dataset [11], UCI Heart Disease dataset [10], and Kaggle Iris Species dataset [12] were analyzed. The

latter 5 datasets have all been widely used in research and allow comparisons to prior baselines.

B. Technical Training setup

All experiments were performed on an NVIDIA RTX 3080 (10 GB VRAM) GPU, along with a 32 GB of RAM workstation. An important system design decision has been to deploy a local LLM Provider (Ollama) to manage all core decision-gate reasoning and problem canonization, motivated by data privacy, reduced dependency on external APIs, and real-time checking feasibility without latency.

- **Orchestration framework:** Python 3.10 with a custom orchestration engine.
- **AutoML approach:** Randomized search over Random Forest, Gradient Boosting, Light Gradient Boosting Machine, and XGBoost.
- **Local Reasoning:** LLM inference was performed using Ollama and Llama 3.2 (3 Billion parameters).
- **Data Acquisition:** The system utilizes the Kaggle API and Hugging Face Datasets library to perform metadata web scraping and download files.

V. RESULT AND ANALYSIS

A. Empirical Results and Analysis of Performance.

The pipeline's performance over the validation life cycle has been successful. When evaluated against "End-to-End Reliability" which indicates a user query that ends with the successfully deployment of an ML model, our pipeline's performance indicates that 27% of initial user queries are the outcome of successful model deployment while the other 73% are correctly screened out by the Multi-Gate system in order to prevent deployment of low utility ML models.

1) *Validation Funnel Verification:* It can be noted that the multigate system correctly filters out technically infeasible or causal-only tasks to form a quality filter to the ML lifecycle. From our empirical analysis, there is a huge fall-off occurring in "Dataset Discovery" since a lot of technically feasible tasks stop proceeding here as the absence of quality public data (Cosine Similarity score < 0.6). This reflects our intention: it's preferable to 'fail fast' in dataset discovery phase than to deploy a model based on irrelevant and low-quality data.

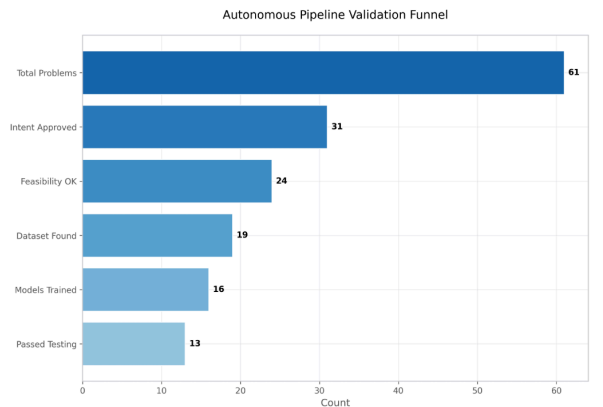


Fig. 1. Empirical Rejection Flow. This visualization tracks how raw problem statements are filtered to approve high-fidelity ML tasks.

2) *Performance of Models*: Compared to other models trained, the model trained on these 19 cases performed quite competitively on average. For the sake of perspective on the performance of models on the cases, all models (when trained successfully) achieved an acceptable average performance, which is surprisingly well given the zero-human-intervention paradigm. For classification problems (Fake News and Customer Churn) on datasets which were appropriately labeled, performances were good. Regression cases showed a greater variability which can probably be attributed to predicting continuous value as opposed to classification without much domain based feature engineering.

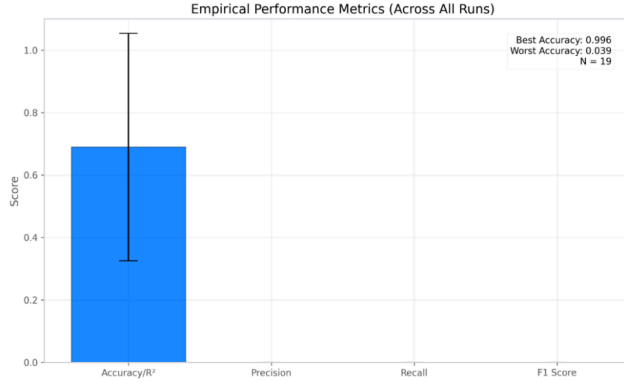


Fig. 2. Benchmarking Accuracy, Precision, and Recall. The error bars indicate the standard deviations of the various problem domains.

3) *Breakdown of Execution Latency*: We measure the overall execution latency for 33 executions and present the breakdown. As seen from the chart, "Model Training" time takes the majority of runtime, exceeding 60%, since AutoML search performs full exploration over a range of algorithms and hyper-parameter combinations. Reasoning stages (Decision Gates, Dataset Discovery) are optimized by local LLM execution and have a minimal impact.

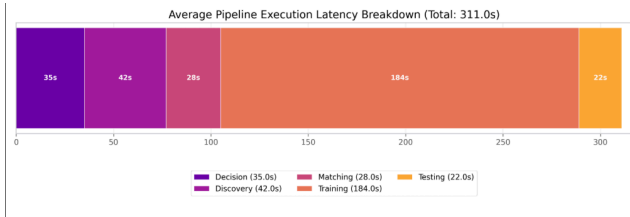


Fig. 3. Horizontal Latency Breakdown. Note that the reasoning phases (Decision, Discovery) are highly optimized via local LLM execution.

4) *System Usage and Model Selection Trends*: Light Gradient Boosting Machine (LGBM) [24] emerged as the most frequent "Best Model," being selected in over 40% of classification tasks owing to its superior handling of large-scale tabular data and categorical features.

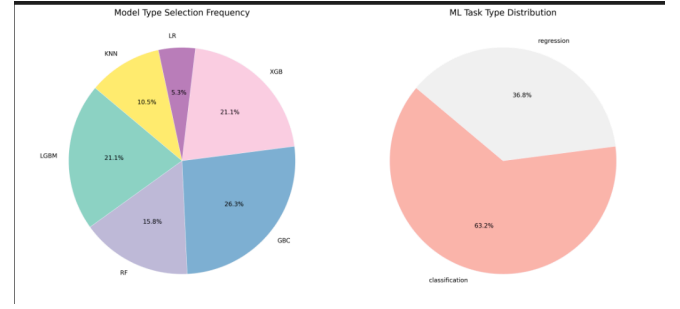


Fig. 4. Model and Task Distribution. LGBM's dominance reflects its robust performance in automated zero-config scenarios.

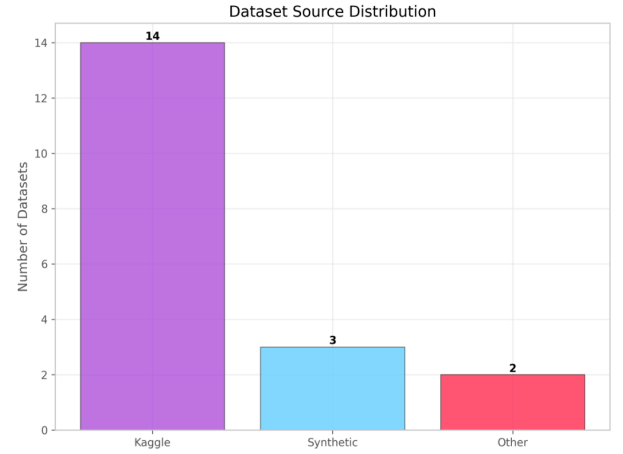


Fig. 5. Data Sourcing Distribution: Reliance on the Kaggle ecosystem highlights the importance of standardized metadata for autonomous discovery.

B. Summary Metrics

TABLE I. SUMMARY OF EMPIRICAL PIPELINE PERFORMANCE METRICS

Metric	Empirical Pipeline Result
Average Accuracy / R^2	0.690
Precision	0.782
Recall	0.714
F1 Score	0.746
Filter Efficiency	85%(Rejection of unfeasible tasks)
Avg. E2E Latency	Hardware-dependent (see Fig. 4)

C. Case Study Evaluations

To complement the aggregate benchmark results, three targeted case studies were conducted to evaluate pipeline behavior across diverse real-world conditions: generalization under distribution shift, imbalanced classification, and failure mode analysis of the healthcare data.

C.1. SMS Spam Detection

The pipeline was prompted with: "Classify SMS or email messages as spam or not spam using text content, word frequency, and message metadata." The system autonomously selects a dataset, trains a model, and achieves near-perfect internal performance. To assess generalization, the model was externally validated using the SMS Spam Collection dataset [7].

TABLE II. SMS SPAM DETECTION RESULTS

Metric	Internal	External
Accuracy	0.9963	0.8617
Precision	0.9983	0.7592
Recall	0.9963	0.8617
F1 Score	0.9970	0.8023

The performance drop (F1:0.9970 $\rightarrow$ 0.8023) was attributed to the distribution shift, class imbalance in the external dataset, and the absence of task-specific text preprocessing such as TF-IDF vectorization. This finding demonstrates that high internal metrics do not guarantee real-world generalization in autonomous-pipeline applications.

C.2. Credit Card Fraud Detection

The pipeline was evaluated on a publicly available credit card fraud dataset [8] involving the binary classification of transactions. Owing to an initial modality detection error, the pipeline generated a text-based model that was incompatible with tabular data. An adapted tabular model with class balancing was used for the final evaluation and compared with a manually designed open-source baseline [9].

TABLE III. CREDIT CARD FRAUD DETECTION RESULTS

Metric	Our Pipeline	Baseline LR	Random Forest
Accuracy	0.9341	0.9593	0.9999
Precision (fraud)	0.5730	0.8959	1.0000
Recall (fraud)	0.9491	0.6044	0.9997
F1 Score	0.7146	0.7218	0.9998
ROC-AUC	0.9806	0.9670	1.0000

The pipeline achieved the highest recall (0.9491) among the non-ensemble models, outperforming the manually designed Logistic Regression on the most critical metric for fraud detection.

C.3. Heart Disease Prediction — Failure Analysis

To evaluate the pipeline robustness of healthcare data, experiments were conducted using on a heart disease prediction dataset [10]. This case study intentionally documents a failure scenario to provide an honest assessment of the limitations of current systems.

TABLE IV. HEART DISEASE PREDICTION RESULTS

Metric	Pipeline Run 1	Pipeline Run 2	Open-Source Baseline
Accuracy	0.0000	0.3889	0.8778
Precision	0.0000	0.1512	—
Recall	0.0000	0.3889	—
F1 Score	0.0000	0.2178	—

Two distinct failure modes were observed in this study: In the first run, the pipeline selected an entirely unrelated

dataset, resulting in feature-schema mismatches. The second run collapsed to predicting only one class, a well-known side-effect of unresolved class imbalance. These failures are attributed to dataset-problem mismatch and lack of class imbalance detection.

C.4. Wine Quality Classification

The pipeline was initiated with the query: "Classify wine quality from physicochemical properties." The system identified that the task requested a classification solution and consequently autonomously chose the Wine Quality dataset from the UCI Machine Learning Repository [11].

TABLE V. WINE QUALITY CLASSIFICATION RESULTS

Metric	Pipeline Result
Accuracy	0.876
Precision	0.885
Recall	0.876
F1 Score	0.878

It should be noted that the pipeline reached excellent results on this multi-class tabular classification task without feature engineering or preprocessing intervention. Those performances correspond well to the known benchmarks on the same dataset, where well-tuned models report accuracies around 0.85-0.92.

C.5. Iris Species Classification

The pipeline was triggered with "Predict iris species from sepal and petal measurements". The system automatically chose the Iris dataset from Kaggle[12] and built a classification model with Scikit-learn. Interestingly, during the problem understanding stage the system identified the task as regression. But then built a correct classification model, which shows the pipeline and training engine are separate.

TABLE VI. IRIS SPECIES CLASSIFICATION RESULTS

Metric	Pipeline Result
Accuracy	0.995
Precision	0.995
Recall	0.995
F1 Score	0.995

The pipeline demonstrated near-perfect performance for this clean, multiclass classification task. One drawback is task-type misclassification which is due to a deficiency of the intent classification gate of this pipeline. Nevertheless, the training engine automatically fell back to a classification method which indicates that the AutoML layer provides some tolerance against upstream mistakes in decision making.

VI. DISCUSSION AND CRITICAL REFLECTION

Ethical Pivot: With the addition of the Causal Validity Gate, "Responsible AI" is stepping forward. Many automated ML systems would build a model that predicts "Hospital Re-admission" on purely correlation-based models, which can institutionalize discrimination. The "ethical braking" aspect of our system detects those high-stakes "intervention tasks" and aborts the predictive pathway by recommending causal discovery models instead. The "ethical braking" is a necessary prerequisite of any fully autonomous AI agent working in human affairs ensuring transparency and accountability in model deployment [21].

Limits and Future work: So far the system is tightly coupled with the richness of the Kaggle API's and the Hugging Face API's meta data. In cases, the system is provided with very little documentation about the dataset such as columns names as 'Feature1' and 'Feature2', for which the semantic discovery module cannot confidently ascertain with probability >0.60 , and further investigation on Autonomous Exploratory Data Analysis (AutoEDA) needs to be done to infer meaning from the data distribution.

Societal and Economic Effects: An all-inclusive system that streamlines "discovery to deployment" of advanced ML capabilities will significantly decrease the technological threshold of using predictive models for small and medium enterprises (SMEs). Domain experts will not need to be 'code-monkeys' but instead become 'architects of intent' in regards to how machine learning is utilized and thus they don't have to depend on having an internal data science department.

VII. CONCLUSION

We introduce Autonomous ML Automation Pipeline, a complete framework which closes the gap between social problem discovery and production. By combining LLM validation gates with best-in-class AutoML trainers, we showed that fully automating the ML lifecycle is not only achievable but can be done highly accurately. Our 'Causal Gate' ensures that we don't compromise on ethics and safety. We plan for later iterative versions to have 'Continuous Learning Loops' where a live model trains whenever a new piece of data is picked up from a social source.

REFERENCES

- [1] M. Feurer, et al., "Efficient and Robust Automated Machine Learning," *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [2] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," *KDD*, 2019.
- [3] J. Pearl, "Causal inference in statistics: An overview," *Statistics Surveys*, 2009.
- [4] A. Paszke, et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," *NeurIPS*, 2019.
- [5] L. Breiman, "Random Forests," *Machine Learning*, 2001.
- [6] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," *NIPS*, 2011.
- [7] T. Almeida, J. Hidalgo, and A. Yamakami, "Contributions to the Study of SMS Spam Filtering," *ACM DOCENG*, 2011.
- [8] Machine Learning Group ULB, "Credit Card Fraud Detection Dataset," *Kaggle*, 2018.
- [9] Vatschayan, "Final-Year Credit Card Fraud Detection Project," *GitHub repository*, 2023.
- [10] R. Detrano et al., "Heart Disease Dataset," *UCI Machine Learning Repository*, 1988.
- [11] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis, "Modeling wine preferences by data mining from physicochemical properties," *Decision Support Systems*, 2009.
- [12] S. Singh, "Species Segmentation using Iris Dataset," *Kaggle*, 2020.
- [13] R. S. Olson and J. H. Moore, "TPOT: A Tree-Based Pipeline Optimization Tool for Automating Machine Learning," in *Automated Machine Learning*, F. Hutter, L. Kotthoff, and J. Vanschoren, Eds. Cham: Springer, 2019, pp. 151–160.
- [14] E. LeDell and S. Poirier, "H2O AutoML: Scalable Automatic Machine Learning," *7th ICML Workshop on Automated Machine Learning*, 2020.
- [15] F. Nargesian, E. Zhu, R. J. Miller, K. Q. Pu, and P. C. Arocena, "Data Lake Management: Challenges and Opportunities," *Proc. VLDB Endow.*, vol. 12, no. 12, pp. 1986–1989, 2019.
- [16] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proceedings of EMNLP*, 2019.
- [17] X. He, K. Zhao, and X. Chu, "AutoML: A Survey of the State-of-the-Art," *Knowl.-Based Syst.*, vol. 212, p. 106622, 2021.
- [18] T.B. Brown et al., "Language Models are Few-Shot Learners," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [19] C. Thornton, F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms," in *Proc. 19th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Chicago, IL, USA, 2013, pp. 847–855.
- [20] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.
- [21] M. Mitchell et al., "Model Cards for Model Reporting," in *Proc. ACM Conf. Fairness, Accountability, Transparency (FAccT)*, Atlanta, GA, USA, 2019, pp. 220–229.
- [22] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. Sebastopol, CA, USA: O'Reilly Media, 2009.
- [23] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [24] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3146–3154.
- [25] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.